

Deep Learning Optimization & Sequence Generation

Comparative Study of Character-Level LSTMs & Batch Normalization

Section: A | Group: G03 (Group 3)

MD Kaif Molla (md.24bcs10221@sst.scaler.com)
Arhan Das (Arhan.24bcs10023@sst.scaler.com)
Sumit Akhuli (Sumit.24bcs10158@sst.scaler.com)
Shiva Gupta (shiva.24bcs10461@sst.scaler.com)

Sarthak Mishra (Sarthak.24bcs10470@sst.scaler.com)
Netram Faran (netram.24bcs10329@sst.scaler.com)
Aditya Prasad (aditya.24bcs10179@sst.scaler.com)

1. Task 1: AI Story Continuation (Project 21)

1.1 Problem Statement

The objective of this project is to model sequential text data at the character level to generate coherent continuations from user-defined prompts. Traditional language models relied on hand-crafted n-gram features or Markov chains, which suffer from the curse of dimensionality and contain very limited context memory windows. This project demonstrates representation learning and sequence modeling by training a Recurrent Neural Network (specifically, an LSTM) to learn the orthographic and grammatical structure of English text from scratch. By training on two highly distinct corpora - William Shakespeare's dramatic plays and Arthur Conan Doyle's Sherlock Holmes detective stories - we analyze how the source corpus style and the decoding temperature affect text generation.

1.2 Technical Approach & Model Architecture

We implement a character-level sequence-to-sequence model using PyTorch:

1. Data Pipeline: The raw text is lowercased and mapped to a unique vocabulary. Sliding input sequences of length $T=100$ characters are extracted with overlapping steps. The targets are the sequences shifted by 1 index.
2. Embedding Layer: Projects discrete character tokens into a dense 128-dimensional continuous vector space.
3. LSTM Recurrent Core: A stacked 2-layer LSTM with 256 hidden units per layer. Gates (Forget, Input, and Output) regulate cell state update equations to maintain long-term gradient flows without decay.
4. Decoding Layer: A fully-connected linear layer projecting the hidden state back to the vocabulary size.
5. Optimization & Training: Trained using the Adam optimizer ($LR=0.002$) and Cross-Entropy Loss over all timesteps. Dropout of 0.2 is applied between LSTMs, and gradient clipping is capped at 5.0 to mitigate exploding gradients.

1.3 LSTM Experiments & Results

Both models were trained for 8 epochs (80 batches per epoch, batch size = 128) using PyTorch MPS GPU acceleration.

- Shakespeare Model: Loss converged smoothly from 2.1700 (Epoch 1) to 0.9955 (Epoch 8).
- Sherlock Holmes Model: Loss converged from 2.3608 (Epoch 1) to 1.1377 (Epoch 8).

We tested the models with a prompt: 'the secret of ' under different decoding temperatures (T):

Greedy (T=0)

Deterministic choice. Leads to repetitive loops:

- Shakespeare: 'the secret of the world is a proper my lord, and my lord, i will not be...'
- Sherlock: 'the secret of his mistle spotting and hadnestrand, however and west...'

Low Temp (T=0.2)

Very safe, but highly repetitive:

- Shakespeare: 'the secret of the world is the world is the world is a proper my lord...'
- Sherlock: 'the secret of his colners was a bother of his columble took...'

Balanced (T=0.6)

Optimal creativity and style mimicry. Correct spelling and syntax:

- Shakespeare: 'the secret of this too, well of parish that's worth's voice...'
- Sherlock: 'the secret of home upon his waiting. he woild dost the band, hereself...'

Extreme (T=1.5)

Uniform probability choice. spelling and coherence disintegrate:

- Shakespeare: 'the secret of cit, fanlerithly to authartial, git dog...'
- Sherlock: 'the secret of yourlecmust. we enkgaiqh, go fors; themes-funl...'

2. Task 2: Batch Normalization Study (Project 23)

2.1 Problem Statement

Deep feedforward networks are difficult to optimize because of Internal Covariate Shift - where intermediate layer activation distributions shift continuously as weights are adjusted - and gradient anomalies (vanishing and exploding gradients). This project evaluates how Batch Normalization stabilizes optimization, accelerates convergence, and provides resilience against aggressive hyperparameters (high learning rates) and near-zero initialization scales.

2.2 Technical Setup & Parameters

We build a 6-layer Deep MLP classifier trained on the MNIST dataset (28x28 grayscale, 10 classes):

- Model Layout: Input (784) -> 5 hidden layers (512 nodes each) with ReLU -> Output (10).
- Setup A (Baseline): Standard Linear layers followed directly by ReLU activations.
- Setup B (Batch Norm): Insertion of a BatchNorm1d layer before each ReLU activation.
- Training details: Trained using mini-batch SGD with a batch size of 128 for 5 epochs. We evaluated three controlled experiments to analyze training dynamics.

2.3 Controlled Stress Tests & Results

Experiment	Configuration	No Batch Norm (Baseline)	With Batch Norm
Exp 1: Standard	LR = 0.01 Kaiming Init	Slow convergence Final Test Acc: 92.83%	Fast convergence Final Test Acc: 97.10%
Exp 2: High LR	LR = 0.2 Kaiming Init	DIVERGED / FAILED Final Test Acc: 11.35%	STABLE / OPTIMAL Final Test Acc: 98.24%
Exp 3: Small Init	LR = 0.01 Std Dev = 0.001	VANISHED / FAILED Final Test Acc: 11.35%	STABLE / RECOVERED Final Test Acc: 97.48%

2.4 Empirical Analysis & Insights

- Experiment 1 (Standard): Batch Norm speeds up learning immediately. By epoch 2, the BN network hits 95.2% accuracy, whereas the baseline lags at 88.6%. The final margin is 97.10% vs 92.83%.
- Experiment 2 (High LR = 0.2): The baseline diverges immediately. Aggressive updates blow up activations, leading to saturated losses (2.3026) and random guessing (11.35%). Batch Norm handles the high learning rate with complete stability, yielding our highest overall score of 98.24% test accuracy.
- Experiment 3 (Small Init = 0.001): Without BN, activation variances decay exponentially across the 6 layers down to zero. Learning stops. Batch Norm recalculates and rescales intermediate variances back to unit scale at each step, allowing the network to recover and achieve 97.48% accuracy.

3. Comparative Synthesis & Core Theory Connections

Our empirical studies connect directly to core Deep Learning Theory:

1. Decoupling Weight Scales: Dividing layer outputs by their standard deviation makes Batch Norm scale-invariant. Scaling weights does not change output distributions or gradient magnitudes, enabling stable high learning rates.
2. Representation Learning: The character LSTM proves that networks can self-assemble hierarchical features (spelling rules, syntactic breaks, and author styles) directly from raw text sequences without word boundaries.
3. Decoupling Logit Entropy: In language modeling, temperature acts as an entropy scale. At low temperature, predictions are deterministic and repetitious. High temperature distributes probabilities evenly, producing high-entropy gibberish. $T=0.6$ acts as the optimal point for vocabulary creativity and formatting consistency.

4. Subgroup Contributions & Tasks

To ensure collaborative productivity, our 7 group members divided code and slides as follows:

- Subgroup Project 21 (LSTM sequence modeling): Developed the embedding-LSTM model code, character mapping functions, and temperature logit decoding loops.
 - * MD Kaif Molla, Arhan Das, and Sumit Akhuli.
- Subgroup Project 23 (Batch Normalization analysis): Built the 6-layer MLP models, coded the SGD optimizer sweeps, and evaluated training logs under high learning rate and poor initialization scales.
 - * Sarthak Mishra, Netram Faran, Aditya Prasad, and Shiva Gupta.
- Subgroup Presentation & Report Design: Co-designed and compiled the final 15-slide PowerPoint deck using python-pptx templates and structured this academic report.
 - * Shiva Gupta and Arhan Das.